

^cCSEN - nvillegaskirchman@scu.edu

In this section, we will explore a variety of CNN-based feature extraction techniques to derive informative representations from gastric histopathology images. The extracted features will then be used to train a baseline XGBoost classifier, allowing us to evaluate and compare the effectiveness of each CNN model in capturing relevant visual patterns for cancer detection. The goal is to identify the most suitable feature extractor that enhances classification performance for distinguishing between different kind of tissues (LYM, TUM, STR,etc..).

The diagram illustrates the AlexNet architecture. It starts with an input image of size $224 \times 224 \times 64$. This is processed by five convolutional layers (conv1 to conv5) and three fully connected layers (fc6 to fc8). The layers are color-coded: blue for convolution+ReLU, red for max pooling, and green for fully connected+ReLU. The dimensions of the feature maps are indicated at each stage.

- conv1:** $112 \times 112 \times 128$
- conv2:** $56 \times 56 \times 256$
- conv3:** $28 \times 28 \times 512$
- conv4:** $14 \times 14 \times 512$
- conv5:** $7 \times 7 \times 512$
- fc6:** $1 \times 1 \times 4096$
- fc7:** $1 \times 1 \times 4096$
- fc8:** $1 \times 1 \times 1000$

Legend:

- convolution+ReLU (blue)
- max pooling (red)
- fully connected+ReLU (green)

	precision	recall	f1-score	support
ADI	0.81	0.81	0.81	777
MUS	0.59	0.57	0.58	777
NOR	0.79	0.83	0.81	778
DEB	0.63	0.61	0.62	778
LYM	0.65	0.62	0.64	778
MUC	0.54	0.54	0.54	777
STR	0.56	0.52	0.54	778
TUM	0.68	0.67	0.63	777
accuracy			0.65	6220
macro avg	0.65	0.65	0.65	6220
weighted avg	0.65	0.65	0.65	6220

Confusion Matrix

	ADI	MUS	NOR	DEB	LYM	MUC	STR	TUM
ADI	626	7	2	82	23	29	0	8
MUS	7	440	61	46	40	59	56	68
NOR	0	49	646	2	2	37	16	26
DEB	94	38	6	478	50	53	21	38
LYM	29	57	5	44	483	38	113	9
MUC	12	66	41	67	18	421	54	98
STR	2	40	39	17	124	48	405	103
TUM	2	50	14	27	3	96	64	521

The diagram illustrates the architecture of the proposed deep learning model. It starts with an **Input Image** of a cell. The image is processed through a series of layers:

- Conv 3x3-BN**: Initial convolutional layer.
- MBlock1, k3x3**: First residual block.
- MBlock2, k3x3**: Second residual block.
- MBlock3, k3x3**: Third residual block.
- MBlock4, k3x3**: Fourth residual block.
- MBlock5, k3x3**: Fifth residual block.
- MBlock6, 3x3-RC**: Sixth residual block with residual connection.
- MBlock7, k3x3-RC**: Seventh residual block with residual connection.
- MBlock8, k5x5**: Eighth residual block.
- MBlock9, k5x5-RC**: Ninth residual block with residual connection.
- MBlock10, k3x3**: Tenth residual block.
- MBlock11, k3x3-RC**: Eleventh residual block with residual connection.

The output of the first path is merged with the output of the second path (which starts with a **MergeConv, k5x5, RC** layer) and then passes through:

- MBlock12, 3x3**: Twelfth residual block.
- MBlock13, 3x3**: Thirteenth residual block.
- MBlock14, 3x3**: Fourteenth residual block.

The final output is processed by a **Conv1x1** layer, followed by a **GAP** (Global Average Pooling) layer, and finally an **FCSoftmax** layer to produce the classification results: **Normal Cancer Cell** and **ALL Cell**.

Santa Clara University June, 2025 Project Check In 1-4

EfficientNetB3 is a deep learning model that, like VGG-166, processes an image through a series of convolutional layers designed to extract features in images. However, unlike VGG-16, EfficientNetB3 uses a special technique called compound scaling, which balances the depth (number of layers), width (number of channels), and resolution (input image size) of the network to improve both accuracy and speed. It also has many more convolutional layers (384 in total) split up into 7 blocks (with each one progressively transforming the image into more abstract and high-level features). Similar to VGG-16, EfficientNetB3 can be used for feature extraction by removing the classification layers and only keeping the base.

For EfficientNetB3, we tried using a feature extraction method similar to the one used for VGG-16. For initial testing, we did not try to fine-tune the model on the dataset beforehand, and just performed feature extraction on the data immediately using EfficientNetB3. The main difference was that we had 1536 features with EfficientNetB3 compared with VGG-166's 512. We then passed in these features into an XGB classifier and got an accuracy of 0.6146, which is somewhat adequate and comparable to the VGG-16 accuracy after fine tuning.

1.2.1. Fine Tuning

For fine-tuning, we used a custom classifier on top of a pretrained EfficientNetB3 model using Keras. Initially, we froze all the layers of the base model and trained only a custom classification head consisting of a global average pooling layer, a dense layer with 256 units, and a dropout layer for regularization. We applied techniques such as rotation, zoom, and horizontal flipping to the data during training to improve generalization. After training the top layers for 5 epochs, we unfroze the last 30 layers (since EfficientNetB3 has more layers than VGG-16) and continued training the full model with 10 epochs and a lower learning rate to fine-tune the pretrained weights. We again included a dropout layer with a rate of 0.5 for regularization and overfit.

Interestingly, fine-tuning the model actually worsened its accuracy, decreasing it down to 0.5045. A possible explanation for this is that fine-tuning the model ends up overfitting it, leading to poorer generalization on actual data. Potential adjustments could be made to parameters such as learning rate and the number of layers unfrozen to potentially combat this problem. Until the right parameter values are found (which might take a while considering that it takes 2 hours to run the fine-tuning), we'll consider the EfficiencyNetB3 without fine-tuning as the best feature extraction method using EfficiencyNetB3, and display the metrics below.

Accuracy: 0.6146

Classification Report:

	precision	recall	f1-score	support
ADI	0.75	0.76	0.75	777
MUS	0.55	0.51	0.53	777
NOR	0.81	0.78	0.80	778
DEB	0.59	0.58	0.58	778
LYM	0.66	0.63	0.64	778
MUC	0.50	0.54	0.52	777
STR	0.54	0.53	0.53	778
TUM	0.54	0.59	0.57	777
accuracy			0.61	6220
macro avg	0.62	0.61	0.62	6220
weighted avg	0.62	0.61	0.62	6220

Figure 4. Best EfficiencyNetB3 Results

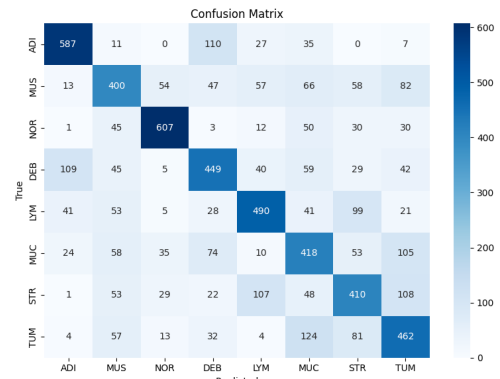


Figure 5. Best EfficiencyNetB3 Confusion Matrix

1.3. Custom CNN-based Feature Extractor

For the custom CNN-based feature extractor, the idea was simple: use torch.nn layers to build a neural network for feature extraction from scratch. This was done by creating a convolutional layer for extracting the features, a pooling layer for reducing the size of each feature, and a normalization layer to prevent high variance. This process was done twice before the data was flattened and stored as a numpy array. However, there were some issues that arose from this such as the time constraints. Currently, the time constraints for extracting the features may be small, but the time needed to convert that numpy array into a format that is usable for a traditional machine learning model does take a lot of time. Furthermore, this transition causes a large loss of data. Even though the data loss during the feature extraction was 0.8, large amounts of the data got lost after translating the features into a different format. This caused accuracy to fall to 0.15 from 0.60 with pre trained models. However, further testing is still being done as since pre-trained models are based off these layers, we feel like the end product from a custom CNN-based feature extractor will yield better results in the long run.

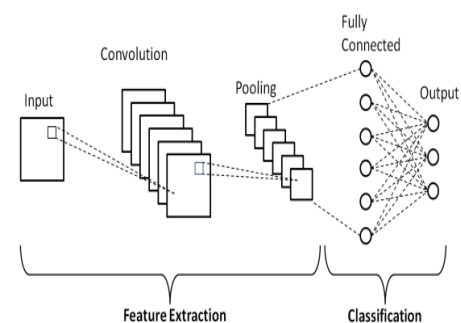


Figure 6. Custom Model Framework

1.3.1. Fine Tuning

For fine-tuning the model, this process was much more complex than the pre-trained models. Because this model is built from scratch, the values have to be manually tweaked which is something that has been done. Originally, the parameters took 3 input features and sent 16 output layers. However, we changed this to 32 and the loss for each epoch of training the data was smaller. Additionally, the kernel size was changed from 3 to 4 which means that the model reads a 4x4 grid rather than a 3x3 grid. This process, did actually seem to increase the loss as there was a wider area to extract features from which left the model more prone to error. Currently, the plan for this is to use variables and biases to automatically tweak the values as the model continues being trained.

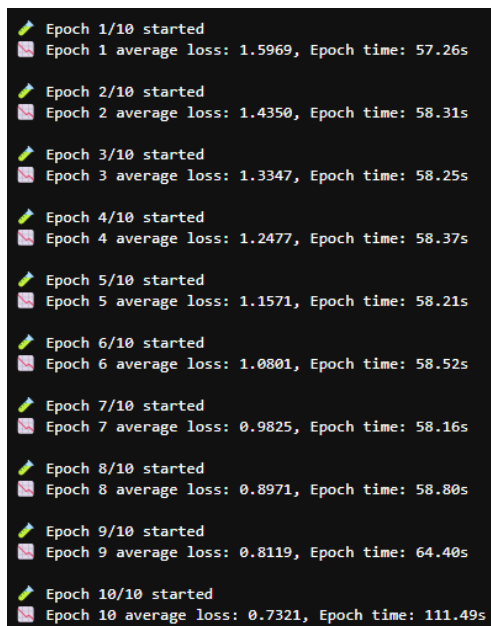


Figure 7. Custom CNN Loss Per Epoch

1.4. Model of Choice

The model that we have decided to use for our feature extraction is VGG-16. The reason we have chosen this model as the accuracy of this model is the highest and has an easy process for fine-tuning.

2. GasHisSDB dataset

The GasHisSDB dataset is composed of image segments derived from high-resolution 2048×2048 Whole Slide Images (WSIs), which are detailed digital histopathology scans of entire gastric tissue samples. These segments are provided at three resolutions—80×80, 120×120, and 160×160 pixels. For this project, we focus exclusively on the 160×160 pixel images, which are subsequently rescaled to 224×224 pixels to match the input size specifications of the HMU-GC-HE-30K dataset. This preprocessing step ensures consistency across both datasets and facilitates model compatibility in the subsequent stages of our pipeline.

Each image in the GasHisSDB dataset is labeled as either normal or abnormal, making it well-suited for the first stage of our proposed two-stage classification model, as described in Section 3.

2.1. Data Visualization and Description

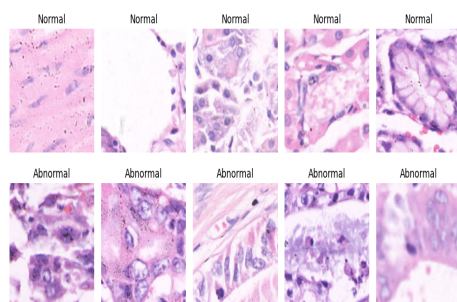


Figure 8. Sample Images

The figure above presents sample images from the GasHisSDB dataset, each accompanied by a corresponding label indicating whether the tissue is normal or abnormal. These labeled examples are valuable for supervised learning, enabling the training of a

reliable classification model in the first stage of our pipeline.

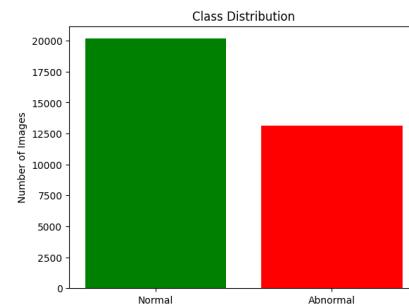


Figure 9. Class Distribution

The subset of the GasHisSDB dataset used in this project comprises a total of 33,285 images, with 20,160 labeled as normal and 13,125 labeled as abnormal. This class distribution will inform the training of our binary classifier in the first stage of the proposed model.

3. Update Proposal

With the discovery of the new GasHisSDB dataset, we propose a two-stage classification pipeline to enhance interpretability in gastric cancer diagnosis. In the first stage, we train a binary classifier on the GasHisSDB dataset to determine whether a given tissue image is abnormal. Upon detecting abnormality, we proceed to the second stage by segmenting the image into smaller patches. Each patch is then analyzed using a model trained on the HMU dataset to classify the specific tissue types it contains (e.g., tumor, stroma, lymphocytes). This hierarchical approach allows us to spatially map the Tumor Microenvironment (TME) within abnormal tissues and investigate how the presence and proportion of various tissue components may influence tumor development and contribute to abnormal pathology.

3.1. General Pipeline

3.1.1. Step 1 - Binary Classification

During this step, we will train a model to classify whether a given gastric tissue image is abnormal or normal. Upon completing this binary classification, we will proceed to segment both abnormal and normal tissue images into smaller patches. Each patch will then be analyzed to identify the specific tissue types present. By examining both classes, we aim to gain deeper insights into how different cellular components and microenvironmental factors—such as the presence of stroma, lymphocytes, or tumor regions contribute to or inhibit tumor development. This comparative approach allows for a more comprehensive understanding of the Tumor Microenvironment (TME) and its influence on tumors progression.

Possible models that we would be using for binary classification are Logistic Regression, XGB classifier, and MLP.

3.1.2. Step 2 - Divide Images into Patches

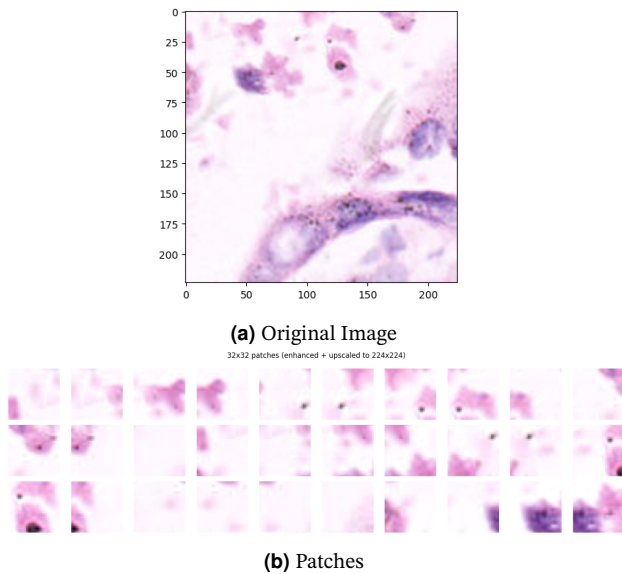


Figure 10. Image Segmentations

As illustrated in the figure above, the original 224×224 tissue image will be divided into smaller patches of size 32×32 pixels. Depending on the stride used during extraction, this process will yield approximately 190 to 200 patches per image.

One limitation of this patch-based approach is the potential reduction in visual clarity, particularly in terms of tissue contrast and stain definition. The downscaling involved in patch extraction can cause the resulting patches to appear faded, making it more difficult to capture fine-grained histological features. To mitigate this issue, we apply Contrast Limited Adaptive Histogram Equalization (CLAHE), a technique that enhances local contrast while preventing over-amplification of noise. This helps preserve critical tissue structures and improves the interpretability of patch-level classifications.

3.1.3. Step 3 - Final Classification

In this step, each individual patch will be passed through a secondary classification model trained on the HMU-GC-HE-30K dataset. This model is designed to classify each patch into one of the eight predefined tissue categories (e.g., LYM, TUM, STR, etc.). Following classification, we will compute the proportion of each tissue class present within both normal and abnormal gastric tissue samples. By aggregating these results across all samples, we will calculate the average percentage composition of each tissue type for both categories. This analysis aims to reveal the microenvironmental differences between normal and abnormal tissues, providing deeper insights into tumor progression and the role of the Tumor Microenvironment (TME).

4. Individual Contribution

Quan Do (Josh) - 33.3%
Andrew Vattuone - 33.3%
Nicolas Villegas-Kirchman - 33.3%

5. References

References

- [1] Gurina, Tatyana S., and Lary Simms. "Histology, Staining." StatPearls - NCBI Bookshelf, 1 May 2023, www.ncbi.nlm.nih.gov/books/NBK557663/?utm_source=chatgpt.com.

- [2] Lou, Shenghan, et al. "A Large Histological Images Dataset of Gastric Cancer With Tumour Microenvironment Annotation for AI." *Scientific Data*, vol. 12, no. 1, Jan. 2025, <https://doi.org/10.1038/s41597-025-04489-9>.
- [3] He, Xiangyang, et al. "Clinical Significance of the Tumor Microenvironment on Immune Tolerance in Gastric Cancer." *Frontiers in Immunology*, vol. 16, Feb. 2025, <https://doi.org/10.3389/fimmu.2025.1532605>.
- [4] Matsuoka, Tasuku, and Masakazu Yashiro. "Molecular Mechanism for Malignant Progression of Gastric Cancer Within the Tumor Microenvironment." *International Journal of Molecular Sciences*, vol. 25, no. 21, Oct. 2024, p. 11735. <https://doi.org/10.3390/ijms252111735>.